

MyFood - Projeto para ser implementado na disciplina de programação 2 UFAL utilizando-se de testes e programação orientada a objetos.

João Victor Duarte do Nascimento

FLUXO E ARQUITETURA DO PROJETO

O projeto não usará um banco de dados relacional tradicional. O fluxo será em memória, ou seja, enquanto o programa estiver rodando, todos os dados serão armazenados na RAM usando coleções do Java. Quando o programa encerrar, as coleções que estão na RAM serão salvas em um arquivo de texto no formato XML, teremos que usar a classe `java.beans.XMLEncoder` e toda vez que o sistema for iniciado, a primeira coisa que ele deve fazer é usar a classe `java.beans.XMLDecoder` para ler o arquivo XML e carregar os dados;

Com a biblioteca `EasyAccept` é possível criar uma ponte entre os arquivos `.txt` que serão testados pelo programa. Então para funcionar é necessário seguir um Design Pattern chamado Facade (fachada) e é importante lembrar que o Facade não deve gerenciar as regras de negócio, ela é uma classe central no programa que contém métodos com as assinaturas exatas exigidas pelos testes. O `EasyAccept` lê o arquivo de texto linha a linha e quando encontrar um comando ele vai procurar e executar o método correspondente dentro da classe Facade.

A estrutura do projeto foi dividida em pastas com as seguintes funções:

- `/`: A raiz do projeto, apenas `Main.java` e `Facade.java`, que são as portas de entrada da aplicação
- `/models`: Estão as entidades e dados puros
- `/controllers`: Onde fica a inteligência e as regras de negócio
- `exceptions`: Tratamento de exceções personalizado
- `/tests`: Arquivos do `EasyAccept`

- docs: Documentação do projeto.

PRIMEIRA FASE DO PROJETO - USUÁRIOS

Toda responsabilidade de gerar IDs únicos estão na classe `ControladorDeUsuarios`, que é gerenciar os usuários, olhando para a coleção `Map<String, Usuario>`. A classe `Usuario` é uma classe abstrata e não deve ser instanciada diretamente, as classes que herdam `Usuario` são `Cliente` e `DonoEmpresa`, cada uma com sua regra de negócio.

Ponto importante: Como estamos usando o decodificador para XML é necessário adicionar construtores vazios e os setters para conseguir retirar o objeto ao ler o arquivo.

Sobre a primeira parte do projeto, vamos começar na pasta `models` com a classe abstrata `Usuario` criada respeitando os moldes do problema e em seguida classes que herdam `Usuario` que são `Cliente` e `DonoEmpresa` com seus respectivos atributos.

Na pasta `controllers` foi implementado a classe `ControladorDeUsuario` que é responsável pela lógica de negócio. Temos alguns métodos importantes:

- `zerar`: zera todos os dados
- `gerarId`: gera os IDs únicos de cada usuário.
- `criarCliente`: cria um novo cliente depois de checar os dados base e verificar email
- `criarDono`: cria um novo dono depois de checar os dados base, verificar email e cpf
- `login`: fazer login no sistema se senha e email estiverem de acordo.
- `getAtributosUsuario`: extrai informações sobre o usuário
- `buscarUsuarioPorId`: usa o ID para encontrar um usuário

- validarDadosBase: verifica a integridade de todos os dados pertinentes ao usuário em questão
- validarCpf: certifica que o CPF segue o padrão de validação
- verificarEmailExistente: analisa se o email já está em uso

Já na raiz, mais precisamente no Facade, esses métodos são chamados seguindo as regras de nomenclatura imposta pelo método avaliativo do professor.

SEGUNDA FASE DO PROJETO - EMPRESAS

Na segunda fase do projeto, segundo os mesmo moldes apresentados, foi desenvolvido a implementação de empresas. Então existe uma classe abstrata chamada Empresa e uma classe que a herda chamada Restaurante que recebem os atributos gerais e específicos para construção do negócio, ambas na pasta models. Para fazer a lógica de controle foi necessário a implementação na pasta controllers a class chamada ControladorDeEmpresa que foi atribuído alguns métodos importantes:

- zerar: zera todos os dados
- gerarId: gera os IDs únicos de cada empresa.
- criarEmpresa: cria uma nova empresa depois de checar os dados necessário respeitando a lógica de negócio
- getEmpresasDoUsuario: Busca todas as empresas que estão atreladas a um usuário do tipo Dono
- getIdEmpresa: verifica se empresa existe na lista de um dono
- buscarEmpresaPorId: Busca empresa por ID único.

Vale lembrar que também foi atribuída à lógica de negócio todo o tratamento de exceções previstos em /exceptions/Empresas. No Facade foram adicionados os métodos do controllers e o método getAtributoEmpresa, pois como o ControladorDeEmpresa não tem acesso aos nomes dos usuários é uma boa prática ajustar a lógica para o controller apenas buscar a empresa e a FAcade fará o

meio-campo pegando o ID do dono e depois vai ao controller de usuário e busca o nome.

TERCEIRA FASE DO PROJETO - PRODUTOS

Partindo para o foco nos produtos, que foi a terceira fase de implementação, foi criada uma classe concreta chamada Produto na pasta models, ela é responsável por abrigar todos os atributos referentes aos produtos de cada empresa. Como nos outros passos, foi feito o tratamento personalizado de exceções presentes em uma subpasta denominada de Produtos em exceptions.

Para controlar toda regra de negócio referentes aos produtos na pasta controllers foi efetuado a adição da classe ControladroDeProduto com os métodos:

- zerar: zera todos os dados
- gerarId: gera os IDs únicos de cada produto.
- criarProduto: cria um novo produto depois de checar os dados necessário respeitando a lógica de negócio
- editarProduto: referente a mudanças em produtos já existentes.
- buscarProdutoPorNomeEEmpresa: o nome já sugere, mas basicamente faz uma busca de um produto usando nome e empresa.
- listarProduto: lista os produtos atribuídos a uma empresa
- validarDadosBase: checa todos os dados base de cada produto.

Além da implementação básica das novas funcionalidades para os produtos no Facade, também foram adicionados os métodos getProduto e listarProduto que, seguindo a ordem, pega os atributos referente a um produto x e lista de acordo com a empresa e produto.

QUARTA FASE DO PROJETO - PEDIDOS

Na última fase da primeira fase do projeto, a implementação seguiu um percurso bastante semelhante ao anterior. Foi usada uma classe concreta para criação dos pedidos, na pasta models com a classe Pedido que possui todos os atributos, getters e setters necessários. Todas as exceções foram tratadas individualmente em exceptions/Pedidos. Além disso, em controllers foram adicionados os métodos lógicos para o melhor controle dos pedidos na classe ControladorDePedidos. Segue os métodos existentes:

- zerar: zera todos os dados
- gerarNumero: gera os IDs (números) únicos de cada pedido.
- criarPedido: cria um novo pedido depois de checar os dados necessário respeitando a lógica de negócio
- getNumeroPedido: através do cliente e empresa busca o número do pedido.
- adicionarProduto: adiciona na lista de pedidos um produto
- fecharPedido: encerra uma lista de pedidos
- removerProduto: remova da lista de pedidos um produto
- buscaPedidoPorId: o nome já sugere que é feita uma busca dos pedidos por id (número).

Por último, foram feitas as alterações no Facade para atender às novas demandas com ênfase no método adicionarProduto que usa tanto a classe Produto quanto Empresa.

QUINTA FASE - MERCADO

A lógica segue bem parecida com a anterior, justamente por seguir um padrão de programação. Mas, foi criada uma classe que herda de Empresa chamada Mercado com seus respectivos atributos próprios, e também foi inserido os tratamentos de exceções esperados para a lógica de negócio, houve uma reestruturação de algumas exceções que serviam para muitos tipos de regras de negócio. A implementação dos métodos foi feita tanto no ControladorDeEmpresa

quanto na Facade, com foco nos métodos que necessitava de lógica que envolvia múltiplas requisições para diferentes controllers.

SEXTA FASE - FARMACIA

Graças ao polimorfismo e a estrutura padronizada do projeto, a única modificação necessária foi a criação de uma nova classe que herda de Empresa chamada Farmacia e adicionar o método criarEmpresa para farmácia no Facade. Todas as exceções e métodos já existiam nos seus respectivos locais.

SÉTIMA FASE - ENTREGADOR

A classe Entregador herda os atributos de Usuário com mais alguns atributos particulares com ênfase no uso de List e ArrayList para armazenar as empresas. As exceções específicas para a lógica foram feitas seguindo a documentação do projeto. A classe ControladorDeUsuariio sofreu upgrade de alguns novos métodos:

- criarEntrega
- mudança no getAtributo

E no Facade alguns métodos, como:

- criarUsuario
- cadastrarEntregador

Os demais métodos já foram criados posteriormente e só foram reutilizados.

OITAVA FASE - ENTREGAS

Na última fase do projeto MyFood, a mais complexa, foi implementada toda a lógica referente à entrega. Foi criada uma classe chamada Entrega com todos os atributos exigidos junto com seus tratamentos de exceções na respectiva pasta. Também foi adicionado um classe chamada ControladorDeEntrega com alguns métodos:

- zerar
- criarEntrega
- buscarEntregaPorId

- getEntrega
- getIdEntrega

No Facade, adicionando métodos que junto aos anteriores criam a lógica necessária para que entregas fossem feitas.